# Problem Set 1: Dimensionality Reduction via Diffusion Maps and Clustering

Geometric and Topological Methods in Machine Learning
CPSC 6440 01 / AMTH 3220 01 / CPSC 4844 01 / MATH 5220 01 / MATH 7440 01

**Assigned**: September 11
**Due**: September 25, 11:59pm
2025

## 1 Introduction

Real world data sets - such as those obtained by high-throughput drug screening or gene sequencing - are massive, containing hundreds of thousands of measured variables of tens of thousands of observations[1]. These data sets often require exploratory analysis. Common questions include

1. Which measured variables are important in understanding my data set (e.g. explain the variance or other statistics of observed data points) and which variables are not so important?
2. Do my observed data points form natural clusters and, if so, which observed data points cluster together and which measured variables best describe these clusters?
3. How can I visualize my observed data points to better understand these relationships and develop a deeper intuition for my data set (whether it be a measured phenomenon or controlled experiment)?

Somewhat ironically, as we develop technologies to measure a larger amount of variables (i.e. when $p$ is large and especially when $p \gg n$), analyzing this high dimensional data and answering these questions above become much more complicated, both statistically and algorithmically. This is known as the *curse of dimensionality*.

In class, we've discussed *dimensionality reduction* techniques to reduce the dimension of data sets from $p$ to $d \ll p$ while (approximately) preserving various structural properties of the data set. Such techniques include Principal Component Analysis (PCA), t-SNE, and Diffusion Maps. Aside from their obvious algorithmic differences, each of these techniques seeks to preserve different properties of the data set and has different mathematical motivations and assumptions. Once we develop a dimensionality reduction technique, how do we - as data *scientists* - experimentally validate these methods? Generally speaking, we first test our method on artificially constructed data sets that we understand either geometrically or statistically (e.g. a low-dimensional point cloud, or points drawn from a known distribution) and then see if our method confirms the known structure in our data set. The next step is to test our method on a real data set that we understand biologically and see if our method confirms the biological understanding.

## 2 Implementing Diffusion Maps

In this assignment, you will implement a Diffusion Map and test this dimensionality reduction method on artificial and real data sets. All code will be written in Python. You are expected to submit a detailed write-up of your experiments, as well as your code, to the Canvas site by the due date of **September 25, 11:59pm** . The layout of the assignment is as follows: Section 2 describes the code specifications that

[1] In this class, we adopt the convention of denoting the number of observed data points by $n$ and denoting the number of measured variables by $p$. Thus, a data matrix $X$ whose rows are observations and whose columns are variables has size $n \times p$.

you will write for implementing the Diffusion Map technique, as well as coding tips. Section 3 outlines the second experiment on a single-cell mass cytometry data set for iPSC Reprogramming. Section 4 contains the submission instructions.

## 2.1 Algorithm Review

The 2006 paper "Diffusion Maps" by Ronald Coifman and Stéphane Lafon [2] is the most cited paper with the Diffusion map algorithm, probably because it provides a very extensive analysis. However, the 2005 NeurIPS paper "Diffusion Maps, Spectral Clustering and Eigenfunctions of Fokker-Planck Operators" is more user-friendly [3] and I recommend reading that paper for more clarification. Let's quickly review the algorithm now. Recall that for data points $X = \{x_1, \dots x_n\}$, kernel matrix $W$ with entries defined by the kernel

$$\kappa(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{\sigma^2}\right)$$

and diffusion parameter $t$, the Diffusion Maps algorithm creates an embedding of the points $\Psi_t : X \to \mathbb{R}^n$ based on the right eigenvectors of the powered Markov transition matrix $P^t$ where $P = D^{-1}W$ and $D$ is the diagonal matrix of row sums. More specifically, let $1 = \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$ be eigenvalues of $P$ with right eigenvectors $\psi_1, \psi_2, \dots \psi_n$.[2] Then the diffusion map is given by

$$\Psi_t(x_j) = \left(\lambda_2^t \psi_2(j), \lambda_3^t \psi_3(j) \dots \lambda_n^t \psi_n(j)\right).$$

Recall that the eigenvector with the eigenvalue 1 is the constant vector; for this reason, we call it the trivial eigenvector and we discard it from our diffusion map since it reveals no information about our data. As it turns out, computing the eigenpairs $\lambda_i$, $\psi_i$ can be done by first computing the eigenpairs of the symmetric matrix $P_s = D^{-1/2}WD^{-1/2}$, denoted $\tilde{\lambda}_i$, $v_i$ and then taking $\lambda_i = \tilde{\lambda}_i$ and $\psi_i = D^{-1/2}v_i$. For somewhat technical reasons, this method is more efficient and numerically stable because $P_s$ is a real symmetric matrix. It is a good exercise to show that, indeed, the eigenvalues of $P$ and $P_s$ are the same and that $D^{-1/2}v_i$ is a corresponding eigenvector $\psi_i$ of $P$, as advertised. We now summarize the Diffusion Maps algorithm below.

**Algorithm 1:** Diffusion Maps Algorithm

**Input:** data set $X = \{x_1, \dots x_n\}$, kernel function $\kappa(\dot{,}\dot{)}$, diffusion parameter $t$
**Output:** diffusion mapping $\Psi_t : X \to \mathbb{R}^n$

1 Construct kernel matrix $W$, $W_{i,j} = \kappa(x_i, x_j)$;
2 Construct symmetric matrix $P_s = D^{-1/2}WD^{-1/2}$;
3 Compute eigenpairs $(\lambda_1, v_1), \dots (\lambda_n, v_n)$ of $P_s$;
4 Compute (normalized) eigenvectors of $P$ as $\psi_i = D^{-1/2}v_i / \|D^{-1/2}v_i\|$;
5 Construct diffusion map $\Psi_t(x_j) = (\lambda_2^t \psi_2(j), \lambda_3^t \psi_3(j) \dots \lambda_n^t \psi_n(j))$;
6 **return** $\Psi_t$

## 2.2 Implementation Details

This assignment will be written using Python. For array computations, you should use the `numpy` package. You may find the following commands particularly useful (some of these are in the `numpy.linalg` subpackage): `sum`, `std`, `exp`, `abs`, `zeros`, `dot`, `norm`, `power`, `svd`, and `eigh`. See the `numpy` documentation online for detailed function descriptions. For plotting, please use the `matplotlib` package, as it is easy to use and interfaces well with `numpy` arrays.

We have provided you with a skeleton file, `ps1_functions.ipynb`, that contains the following empty functions which you should complete. The purpose of each function is briefly stated in the function header, but you should review the Diffusion Maps method to remind yourself how these functions should fit together. The distance matrix calculation and the eigendecomposition are the most computationally expensive components of the algorithm; we have designed the functions in the skeleton file accordingly. The skeleton file

[2] It shouldn't be immediately obvious that this non-symmetric matrix has positive real eigenvalues, with the largest one being equal to 1. This can be shown in a few ways, but the most popular way is through the Perron–Frobenius theorem. Proving this would be a good exercise in linear algebra.

`ps1_notebook.ipynb` contains a function `read_json_files()`, which can be used for loading data. You may add more functions to `ps1_notebook.ipynb` if you feel that they will help you in your experiment. All code should be sufficiently commented so that someone who understands diffusion maps [3] can easily understand your code. We highly recommend using Google Colab for this assignment and here is a link to the `ps1_notebook.ipynb`: **Notebook Link**. Please first **save a copy in your drive** by going to File → Save a copy in Drive and then fill in the notebook in your saved version.

In Lectures, we introduced PCA as providing $p$ linear combinations of features that best explain the variance. To obtain the projected data points, we can evaluate the linear combinations of the features of each data point; however, this is not the most efficient computational method. We mentioned that the Singular Value Decomposition (SVD), which decomposes a data matrix $X = USV^T$ is an appropriate way to obtain both the linear combination of features or loadings (which are given in $V$) *and* the projection of the data points. (which are given as $US$). When writing your code, you should always center the data and compute its SVD rather than constructing the covariance matrix and computing its eigendecomposition. For computing diffusion maps, it is best to use `eigh`, for reasons described above.

**Question 2.1.** *Prove that 1 is an eigenvector of $P$. Show its associated eigenvalue.*

**Question 2.2.** *Prove using the spectral theorem that $P$ has an eigendecomposition with real eigenvalues $P = U\Lambda U^{-1}$.*

**Question 2.3.** *Prove that the embeddings generated by Diffusion Maps preserves diffusion distance on the data manifold. In particular, if $\Psi_t(x)$ denotes the diffusion map at time $t$, then for any two points $x$ and $y$ in the dataset, show that the diffusion distance:*

$$D_t^2(x,y) \;=\; \sum_{z\in\mathcal{X}} \frac{\big(p_t(x,z) - p_t(y,z)\big)^2}{\phi_0(z)}$$

*can be expressed equivalently as the squared Euclidean distance in the diffusion map space as:*

$$D_t^2(x,y) \;=\; \big\|\Psi_t(x) - \Psi_t(y)\big\|^2$$

# 3 Experiment: iPSC Reprogramming Dataset

This is a mass cytometry dataset taken from [4] that shows cellular reprogramming from mouse embryonic fibroblasts (MEFs) to induced pluripotent stem cells (iPSCs) via the introduction of a transcription factor known as Oct4. This data set measures 33 proteins at the single-cell resolution. The protein markers measure

1. pluripotency or stem-ness, e.g. sox4 oct4
2. differentiation: e.g. nanog and lin28 which are embryonic stem cell (ESC) markers
3. cell-cycle: e.g. pRB which is a cell proliferation marker
4. signaling status: e.g. pakt which is a growth inducing signal and p53 which signals DNA damage and could induce cell cycle arrest. The "p" preceding some of these markers indicates that the measurement is for the phosphorylated (usually active) conformation of the protein
5. apoptosis: e.g. ccasp3 which is a caspase marker involved in programmed cell death

Additionally a "timepoint" marker shows the day after stimulation with reprogramming factors that the measurement was taken from. Here you have a mix of timepoints. These are low-granularity time points (on the order of days), while the actual transition is much finer grained. These protein markers enable the measurement of the reprogramming status. For instance, a subset of cells shows evidence of being early reprogramming intermediates with the "correct" set of reprogramming factors Sox2+Oct4+Klf4+Nanog+, another subset seems successfully reprogrammed with ESC-like lineages expressing markers such as Nanog, Oct4, Lin28 and Ssea1, and Epcam that are associated with transition to pluripotency.

[3] for instance, the course grader

As before, use the provided `read_json_files()` for reading the data set `ipsc_data_set.json` and the channel names `channel_names.json`. The original data set contained observed 220,450 time points; however, for computational purposes, we have subsampled the data set to contain 2,000 time points. The *timepoint* of data point is simply its row number. So the first timepoint is row one, the second timepoint is row two, and so on. A variety of other pre-processing steps have already been done for you.

## 3.1 Visualizing Data with Diffusion Maps

Now let's do a similar analysis using Diffusion Maps and see what changes. Here, your task will be to

1. Construct the affinity matrix of the iPSC data set using euclidean distance and adaptive $k$-nearest neighbors Gaussian kernel with $k = 2$ and diffusion parameter $t = 1$.
2. Create two-dimensional scatter plots of the diffusion mapping using different coordinates.
3. Create three-dimensional scatter plot of the diffusion mapping using the first three coordinates for plotting. Color the points with their corresponding timepoint values.
4. Plot the associated eigenvalues of the Markov matrix.
5. Compute the top 5 channels that have the highest absolute correlation with the first, second, and third diffusion components.

**Question 3.1.** *How do the Diffusion Map visualizations look? Are they capturing the time progression of the data set? Do you see any clusters forming? What is the intrinsic dimensionality given by Diffusion Maps? Which channels are most highly correlated with diffusion components? Any guesses to the biological interpretation?*

## 3.2 Changing Parameters

In this section, we'll examine how changing parameters affects the iPSC data set. Your tasks will be

1. Try two different values of diffusion parameter $t$ and visualize the embeddings and the eigenvalues.
2. Try two different values of $k$ for the adaptive kernel and visualize the embeddings and the eigenvalues.
3. For every choice of parameters above, compute the top 5 channels that have the highest absolute correlation with the first, second, and third diffusion components.

**Question 3.2.** *Did the choice of $t$ affect the embeddings and, if so, how? Do the corresponding eigenvalues support your findings? How did the choice of $k$ affect the embeddings and, if so, how? Did previously observed trends (i.e. clusters, time progression) in diffusion dimensions change dramatically? Did the correlated channels change with $\sigma$? How would you interpret this in terms of the data?*

## 3.3 Comparing visualizations

Lets compare the diffusion maps visualization to that of PHATE and tSNE

1. Run PHATE and tSNE on the on the processed iPSC data set singular values.
2. Plot the iPSC data set in two dimensions, using PHATE and tSNE color them using the time steps.

**Question 3.3.** *How do the PHATE and tSNE visualizations compare to the diffusion maps visualization? Are they capturing the time progression of the data set? Do you see any clusters forming?*

### 3.4 Clustering the iPSC dataset with Louvain

The Louvain algorithm [1] is a community detection method that partitions the graphs into clusters by optimizing modularity. Modularity measures how densely edges within a cluster are connected compared to what is expected at random and is defined as:

$$Q = \frac{1}{2m} \sum_{i,j} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{1}$$

where:

- $A_{ij}$ is the weight of the edge between nodes $i$ and $j$,
- $k_i = \sum_j A_{ij}$ is the degree (sum of edge weights) of node $i$,
- $m = \frac{1}{2} \sum_{i,j} A_{ij}$ is the total number of edges (or total weight of all edges),
- $c_i$ is the community assignment of node $i$,
- $\delta(c_i, c_j) = 1$ if $c_i = c_j$, and 0 otherwise.

The Louvain algorithm operates in two iterative phases:

**Phase 1 Finding Communities:** First, each node is placed in a community. Then, for each node, the algorithm considers moving it to a neighboring community if doing so increases modularity. The evaluation is local and only checks $k$ neighbors of the node, which makes it computationally efficient.

**Phase 2 Coarse Graining:** Once Phase 1 converges, a new graph is constructed where each community discovered in the first phase collapses into a node. Edges between these new nodes are weighted by the sum of the edge weights between the nodes of the communities in Phase 1. Phase 1 is then applied on this reduces graph which captures the higher-level structure of the network.

The two phases are iteratively applied until modularity converges. In single-cell analysis, Louvain clustering is often applied to affinity graphs of cells to discover cell states or lineages.

Your task is to apply the Louvain algorithm to the iPSC dataset:

1. Construct a k-nearest neighbor graph from the iPSC data using Euclidean distances between cells.
2. Apply the Louvain algorithm to cluster cells based on this graph.
3. Report the number of clusters found and approximate size of each cluster.
4. Create a two-dimensional scatter plot using your diffusion map embedding from earlier and color the points by their Louvain clusters.

### 3.5 Comparing clustering methods

We will next cluster this dataset using k-means and spectral clustering.

1. Use the diffusion components you computed in part 3.1 of this homework.
2. Draw a screeplot (a screeplot is a plot that shows the eigenvalues of the diffusion operator sorted in descending order) of the eigenvalues of the diffusion operator. On the x-axis, you plot the component number and on thr y-axis, you plot the magnitude of the corresponding eigenvalue. Each eigenvalue corresponds to the amount of variance explained by its associated diffusion component.
3. Choose an appropriate k, for the k-means.
4. Apply k-means to the raw data, as well as to k diffusion components.
5. Visualize the clusters as a coloring on top of the visuals you developed in part 3.3.

**Question 3.4.** *How do the 3 different clustering methods compare (Louvain, K-means, Spectral Clutering)? Which is following the shape of the data the best? Does one of the clustering methods "match" well with any of the visualizations?*

**Question 3.5.** *Derive the incremental update to the modularity equation when a point moves from one module $i$ to another $j$. The Louvain algorithm optimizes the modularity function defined in Equation 1.*

# 4 Submission Instructions

Each student should submit the following files to **Gradescope** by **September 25, 11:59pm** :

1. A detailed write-up in pdf form, titled `[last name]_[first name]_ps1_report.pdf`
2. A pdf of the notebook with the code and all the outputs including visualizations, titled `[last name]_[first name]_ps1_notebook.pdf`
3. A .ipynb file of the filled in notebook, titled `[last name]_[first name]_ps1_notebook.ipynb`

# References


[1] Vincent D Blondel et al. "Fast unfolding of communities in large networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (Oct. 2008), P10008. ISSN: 1742-5468. DOI: 10.1088/1742-5468/2008/10/p10008. URL: http://dx.doi.org/10.1088/1742-5468/2008/10/P10008.

[2] Ronald R Coifman and Stéphane Lafon. "Diffusion maps". In: *Applied and computational harmonic analysis* 21.1 (2006), pp. 5–30.

[3] Boaz Nadler et al. "Diffusion maps, spectral clustering and eigenfunctions of Fokker-Planck operators". In: *Advances in neural information processing systems*. 2006, pp. 955–962.

[4] Eli R Zunder et al. "A continuous molecular roadmap to iPSC reprogramming through progression analysis of single-cell mass cytometry". In: *Cell Stem Cell* 16.3 (2015), pp. 323–337.